

Nanoleaf Shapes Interface Board

A bus-powered half-duplex interface between any 3.3 V microcontroller and the Nanoleaf Shapes panel bus — taking 42 V, ground and data from a single panel connector.

BUS	TOPOLOGY	INPUT	RAILS OUT
1 Mbaud 8N1	Single-wire half-duplex	42 V from panel	5 V · 3V3
ACTIVE PARTS			
3			

This is a two-block board: a wide-input buck that steals the panels' own 42 V rail so the controller needs no second power supply, and a tri-state transceiver that turns a normal microcontroller UART into the single-wire half-duplex bus the panels speak.

The bus half is a reconstruction of Christian Panton's Half Duplex Interface Board from a photograph of the assembled PCB, cross-referenced against his published protocol notes. The power half is new — his board deliberately omitted it, taking 3.3 V from the host instead.

Every substantive claim below carries its provenance, because a good part of this is inference and you will be putting 42 V near a microcontroller:

PUBLISHED stated in Panton's protocol writeup

INFERRED read off the board photo

DESIGN this document's own choice

Power domain

42 V → 5 V → 3V3

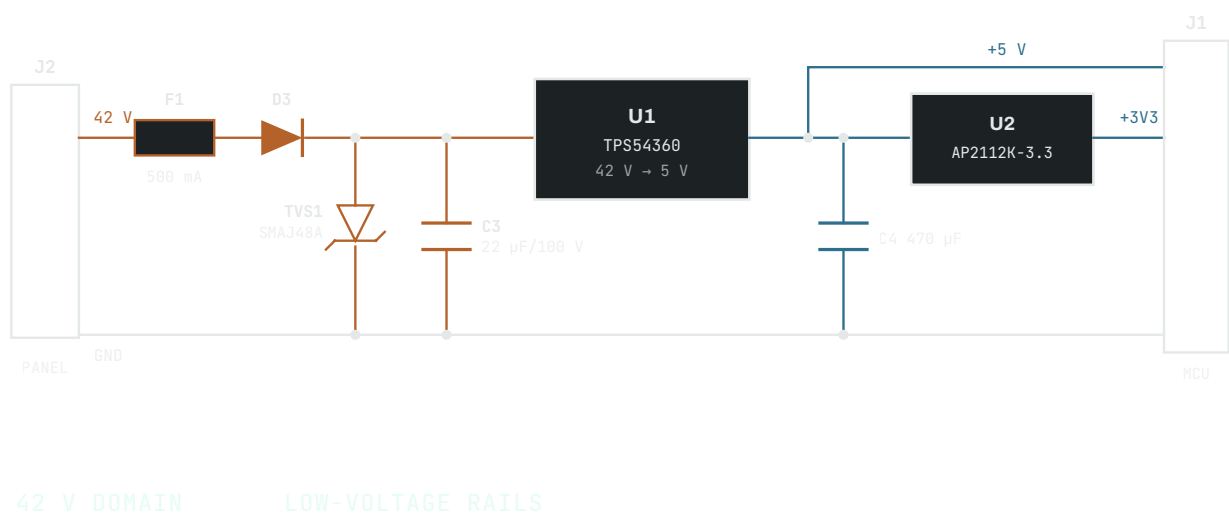


Figure 1 — Power chain. The panel's own 42 V rail feeds a fuse, a reverse-blocking diode and a transient suppressor before reaching the buck. The colour boundary at U1 is the boundary you should also enforce physically on the PCB: nothing on the right-hand side of it should share copper, creepage or a connector with the left.

Why two stages rather than one

A single 42 V → 3.3 V buck is possible — the duty cycle works out around 8%, which at 300–400 kHz is a ~200 ns on-time, inside the TPS54360's minimum. But going through 5 V first gets you a 5 V rail for free, which you want the moment you plug in an ESP32 DevKit board rather than a bare module, and it keeps switching noise a stage further from the bus receiver.

The LDO's dissipation is the thing to sanity-check. An ESP32 averages around 80–120 mA, so $(5 - 3.3) \times 0.1 \approx 0.17$ W — fine in SOT-23-5. The 500 mA WiFi transmit bursts are microseconds long and should be absorbed by C4, not the regulator. If you populate a bare WROOM module and see brownouts, that bulk cap

is the first thing to grow.

LOWER-RISK ALTERNATIVE FOR THE FIRST SPIN

If you would rather not do inductor selection and hot-loop layout on a board whose real purpose is protocol work, replace U1 with a **Recom R-78HB5.0-0.5** — a 72 V-input, 5 V, 500 mA switching module in a 3-pin SIP that needs nothing but input and output capacitors. It costs a few euro more and removes an entire category of first-spin failure. Check its current rating against your final load before committing.

Inrush

You are hot-plugging capacitance onto a live 42 V bus shared with the panels. Keep C3 modest, and be aware that ceramics lose most of their value at rated voltage — a 10 μF /100 V X7R part at 42 V is realistically 2–3 μF . That is why C3 is specified as electrolytic. If the controller resets other panels when you plug it in, add an NTC or a soft-start.

Bus interface

SINGLE-WIRE HALF-DUPLEX

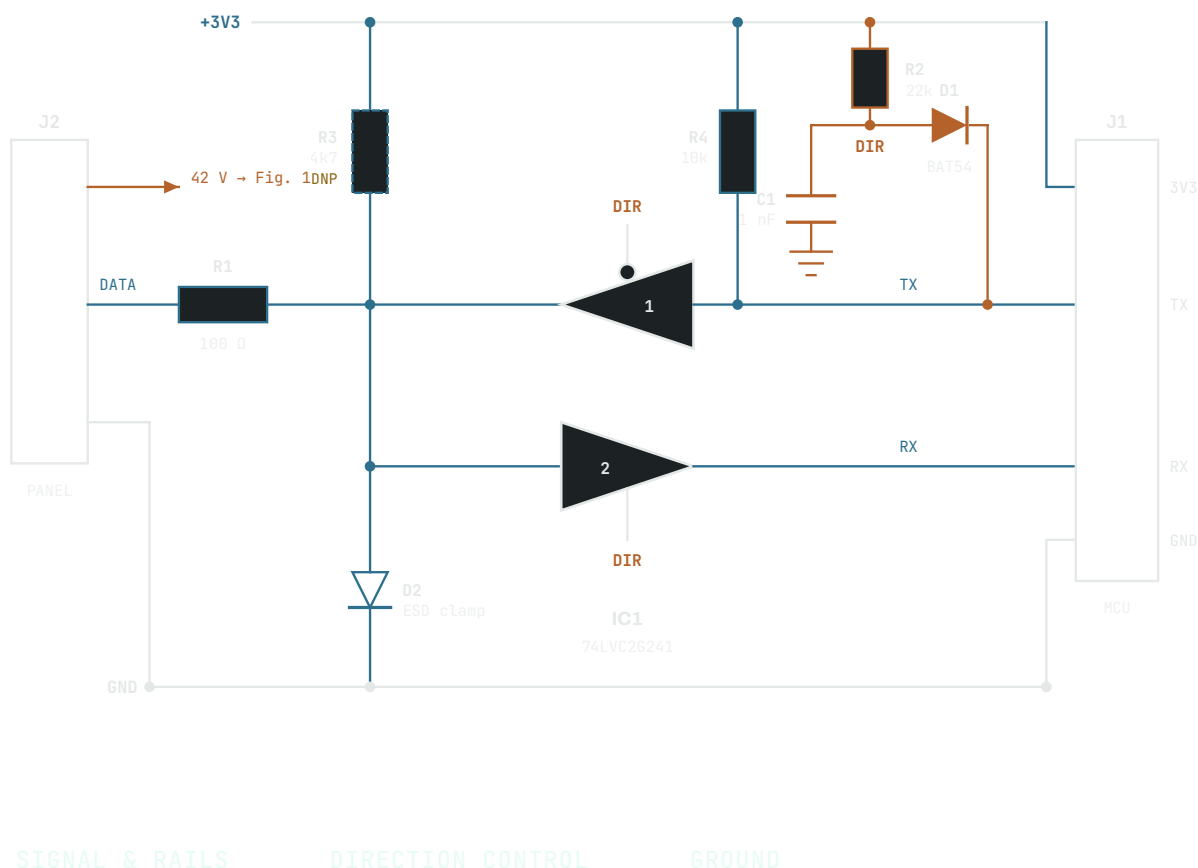


Figure 2 — Half-duplex transceiver with self-timed direction. Buffer 1 drives the bus from the MCU's TX pin; buffer 2 returns the bus to RX. Their enables are complementary, so exactly one is live at any moment and the pair never contend. DIR is not driven by a GPIO — it falls the instant TX does, through D1, and recovers on R2·C1 after transmission stops.

How the direction circuit works

PUBLISHED Panton describes the switching as automatic, with "no separate direction control signal required". **INFERRED** The D1/R2/C1 cluster beside the DIR pad on his board is the standard retriggerable one-shot used in USB-to-RS485 adapters, and it produces exactly that behaviour.

At rest, TX idles high, D1 is reverse-biased, and R2 holds C1 charged — so DIR sits high and the board is listening. The first falling edge of a start bit pulls DIR down through the Schottky almost instantly, enabling the driver. Every subsequent falling edge re-clamps it. When transmission ends, R2 recharges C1 and the board returns to listening on its own.

The only real design constraint is that the hold time must outlast the longest run of high bits *inside* a transmission, or the driver will drop out mid-frame:

```
// worst case: byte 0xFF = start(0) + eight 1s + stop(1) t_high_max = 9
bits × 1 μs = 9 μs // recovery to the logic threshold, from Vf(D1) ≈ 0.3 V
t = R2·C1 × ln( (3.3 - 0.3) / (3.3 - 1.65) ) t = R2·C1 × 0.598 // with R2 =
22 kΩ, C1 = 1 nF t = 22 μs × 0.598 = 13 μs ✓ > 9 μs // turnaround after the
final stop bit 13 μs - 9 μs = ≈ 4 μs
```

Thirteen microseconds clears the worst-case byte with margin and still releases the bus about four microseconds after the last stop bit — fast enough that a panel answering in its polling slot will not be stepped on. **DESIGN** Put R2 on a footprint you are willing to swap: 15 k gives 9 μs and is too tight, 33 k gives 20 μs and is safer if turnaround proves not to matter. This is the one value worth confirming on a scope before you trust the board.

R4 IS NOT OPTIONAL

An ESP32's GPIOs float during reset and for the first moments of boot. If TX floats low, D1 pulls DIR low, buffer 1 enables, and your board drives an undefined level onto a bus shared with every panel in the string — jamming all of them, intermittently, in a way that will look like a protocol bug for a very long time.

R4 holds TX at idle-high whenever nothing is driving it, so the board powers up listening. Panton's board may or may not have this; I could not resolve it from the photo. Fit it regardless.

The pull-up you probably should not fit

R3 is drawn dashed because it is DNP — do not populate — until you have measured. On a shared open-drain bus the panels themselves very likely provide the pull-up, and adding a second one in parallel stiffens the bus and makes it harder for any device to pull a clean low. Fit R3 only if measurement shows the bus floating with no bias of its own.

If push-pull turns out to be wrong

INFERRED A tri-state buffer drives both directions actively, which is fast and clean at

1 Mbaud but will contend if a panel ever pulls low while you are driving high. R1 at 100 Ω limits that contention current to a survivable level rather than preventing it.

DESIGN Worth hedging: lay out an alternate footprint for a 2N7002 in open-drain across the bus node, so if you observe contention you can depopulate buffer 1's output path and switch to open-drain without a respin. Remember that an N-FET inverts — handle it with `uart_set_line_inverse()` rather than in software.

Bill of materials

REV A · 3 ACTIVE PARTS

POWER DOMAIN			
REF	VALUE / PART	PACKAGE	NOTES
F1	500 mA, ≥ 63 V	1206	Slow-blow. Fails before your board does.
D3	STPS2H100 or any ≥ 100 V / 1 A	SMB	Reverse-polarity block. Drop is irrelevant here.
TVS1	SMAJ48A	SMA	48 V standoff. Confirm against your measured rail.
C3	22 μ F / 100 V	Electrolytic	Not ceramic — DC bias derating would eat most of it.
U1	TPS54360	S0-8 PowerPAD	60 V, 3.5 A. Set 5 V with 52.3 k / 10 k feedback divider (V_{ref} 0.8 V).
L1	47 μ H, ≥ 1 A shielded	—	Starting value; confirm with the datasheet or WEBENCH at your chosen f_{sw} .
C4	470 μ F, 10 V	Electrolytic	Absorbs ESP32 WiFi transmit bursts. Grow this if you see brownouts.
U2	AP2112K-3.3	S0T-23-5	600 mA LDO. ~ 0.17 W at typical load.

BUS INTERFACE			
REF	VALUE / PART	PACKAGE	NOTES
IC1	74LVC2G241	VSS0P-8	Dual buffer, complementary enables. Tie 10E and 20E

together as DIR.

R1	100 Ω	0603	Series into bus. Damps ringing, limits contention current.
R2	22 k Ω	0603	DIR recovery. TUNE ON SCOPE
R4	10 k Ω	0603	TX idle-high during MCU boot. DO NOT OMIT
R3	4.7 k Ω	0603	Bus pull-up. DNP UNTIL MEASURED
C1	1 nF C0G	0603	DIR timing. C0G for stability, not X7R.
C2	100 nF X7R	0402	IC1 decoupling, close to the pin.
D1	BAT54	SOD-323	Schottky, cathode toward TX. Low V_f matters here.
D2	PESD3V3L1BA	SOD-323	ESD clamp on the bus node.
Q1	2N7002	SOT-23	Open-drain alternative. Footprint only — see above.

Measure before you order

FIVE UNKNOWNNS

Each of these changes a value on the board. All five can be answered in one session with a multimeter, a logic analyzer and a flex linker cut in half.

1 Actual bus idle voltage

Confirm the data line really is 3.3 V logic and not something that needs level shifting.

Decides → whether IC1 can sit directly on the bus at all

2 Does the bus already have a pull-up?

Measure resistance from DATA to the 3.3 V rail unpowered, and watch the rising edge on a scope.

Decides → fit or omit R3

3 Real rail voltage and ripple

"42 V" is nominal. Measure it loaded, with panels active.

Decides → TVS1 standoff and C3 voltage rating

4 Bit period on DATA

Confirm 1 μ s rather than trusting the writeup. A DMM will not tell you this; use the analyzer.

Decides $\rightarrow$ the whole R2·C1 timing calculation

5 Headroom on the 42 V supply

Measure current drawn by your panel count against the PSU's rating.

Decides $\rightarrow$ whether bus-powering the controller is safe at all

Deliberately out of scope

REV A BOUNDARIES

No microcontroller on board. J1 is a plain 4-pin header. Keeping the ESP32 external for rev A means you can swap boards, use a DevKit's USB and serial console while debugging, and avoid respinning the interface every time you change your mind about the MCU. Integrate later once the bus half is proven.

No edge pads yet. J2 is a 3-pin header intended to meet a flex linker cut in half and stripped — a factory-correct mechanical interface for the price of a spare part. Native edge pads matching the panels' own 10 $\times$ 1.5 mm ENIG geometry are what a finished, shareable controller wants, so that people connect it with a stock linker rather than butchering one. That is a later spin, and it depends on measuring panel PCB thickness first.

No firmware assumptions. The board presents an ordinary UART. Drive it with the ESP32's hardware UART at 1 Mbaud rather than bit-banging through RMT — at that rate you get exactly one RMT tick per bit and no timing margin at all. You will receive your own transmissions as echo; discard them, and treat any mismatch between sent and read-back as free collision detection.

Protocol figures derived from Christian Panton's *Nanoleaf Shapes deepdive, part 1* (January 2025); interface topology reconstructed from a photograph of his *Half Duplex Interface Board*, JLCPCB batch 9097271A_Y1_250103.

